

Gitleaks – Diagnostic et résolutions

- **Faux positifs sur fichiers générés (autogénérés)**

Message d'erreur typique :

```
csharp
```

Copy

```
[INF] scan completed ...
```

```
[WRN] leaks found: 2, Error: Process completed with exit code 1.
```

Par exemple, Gitleaks peut détecter un token générateur dans un fichier auto-généré (RcppExports.R/Cpp) comme un secret. Ici la détection déclenche un code de sortie 1.

Cause : Gitleaks trouve un motif (p. ex. "Generator token") dans du code généré, d'où faux positif.

Reproduction : exécuter `gitleaks detect --no-git --source .` sur un projet contenant du code généré contenant des chaînes de type secret.

Solution concrète : ignorer le secret via le fichier `.gitleaksignore` avec l'empreinte (Fingerprint) du leak. Par exemple :

```
makefile
```

Copy

```
R/RcppExports.R:generic-api-key:2
```

```
src/RcppExports.cpp:generic-api-key:2
```

Cela exclut spécifiquement ces occurrences. (*Note: la fonctionnalité `#gitleaks:allow` existe, mais ne s'applique pas aux fichiers générés.*) Cette fonctionnalité est « expérimentale ».

Bonnes pratiques : éviter de committer des secrets (même générés), exclure/filtres les fichiers auto-générés du scan (par config ou options CI), utiliser `.gitleaksignore` pour les exceptions connues.

- **Fuites de secrets dans l'historique Git (anciennes commits)**

Problème : Après avoir corrigé une fuite, Gitleaks fait toujours état d'anciennes occurrences.

Symptôme : Rapport de scan incluant des leaks dans des commits résolus.

Cause : Gitleaks scanne tout l'historique par défaut. Les anciens commits contenant des secrets persistants déclenchent encore le scan.

Solution :

1. **Supprimer définitivement les leaks du dépôt** via un outil comme [BFG Repo-Cleaner] pour purger les commits anciens.

2. **Configurer l'exclusion via gitleaks** : ajouter dans .gitleaks.toml une section rules.allowlist pour ignorer des commits (ou des scans antérieurs). Par exemple:

toml

Copy

[rules.allowlist]

description = "ignorer anciens commits corrigés"

commits = ["<CommitSHA1>", "<CommitSHA2>"]

ou utiliser un .gitleaksignore pour chaque fingerprint détecté.

Prévention : Ne pas committer de secrets dans Git ; utiliser des secrets managers ; faire des scans locaux avant commit.

- **Règles par défaut non exhaustives (fuite non détectée)**

Problème : Un mot de passe ou token légitime n'est pas signalé par Gitleaks.

Symptôme : Pas de rapport d'erreur malgré la présence d'un secret.

Cause : Le set de règles par défaut de Gitleaks (motifs regex) ne couvre pas tous les formats. Par exemple, un password custom peut échapper aux règles standards.

Solution : Personnaliser la configuration gitleaks. Dans .gitleaks.toml, ajouter au bloc [extend] la ligne suivante pour désactiver les règles par défaut et n'utiliser que vos règles personnalisées:

toml

Copy

[extend]

useDefault = false

Il faut alors définir toutes les règles voulues explicitement. Vous pouvez aussi désigner votre config via l'environnement (ex. export GITLEAKS_CONFIG=/path/to/gitleaks.toml).

Observation : Un bug rapporté indiquait que même après avoir corrigé un secret (ajout puis retrait), Gitleaks ne le redétecte plus, suggérant une forme de **mise en cache interne**. Si cela se produit, utilisez la version la plus récente de Gitleaks ou redémarrez le scan sur un nouveau clone.

Bonnes pratiques : revoir périodiquement les règles par défaut ; ajouter des règles personnalisées pour les formats propres à votre code. Tenir Gitleaks à jour pour bénéficier des nouvelles règles.

- **Erreur « Too many open files » en mode --no-git**

Message d'erreur :

perl

Copy

ERRO open .git/objects/...: too many open files

```[object Object].

**\*Cause\*** : En mode `--no-git``, Gitleaks parcourt le répertoire `.git`` et ouvre un grand nombre de fichiers internes de Git, dépassant la limite OS.

**\*Reproduction\*** : Dans un dépôt local, exécuter `gitleaks --path=. --no-git`` sur un projet de taille conséquente (p. ex. dépôt Rails)[object Object].

**\*Solution\*** :

- **\*\*Ne pas utiliser `--no-git`` quand ce n'est pas nécessaire ; par défaut Gitleaks n'a pas besoin de lire `.git``.**

- **\*\*Exclure explicitement `.git`` : déplacer temporairement ou ignorer ce dossier.**

- **\*\*Augmenter la limite de fichiers ouverts\*\* via l'OS (`ulimit``).**

- **\*\*Mettre à jour Gitleaks (`>=v8`)\*\*** : depuis v8, Gitleaks utilise `git log`` sous le capot (ligne de commande) au lieu d'explorer `.git`` par programme[object Object], ce qui réduit l'usage de fichiers.

**\*Bonne pratique\*** : scanner uniquement le code source (fichiers de travail), pas la base de données Git. Configurer éventuellement `--path`` sur `.`` et éviter `--no-git``.

- **Format de rapport et options CI**

Gitleaks retourne exit code = 1 en cas de fuites détectées (sauf mode de soft-fail explicite). En CI (GitHub Actions, Jenkins, GitLab CI), ajuster la commande selon besoin :

- *Exemple commande CLI :*

bash

Copy

gitleaks detect --source . --verbose

- **Format de sortie JSON/SARIF** : utilise `--report-format [json|sarif]` pour intégration (auto détection par GitLab/GitHub).
- **Authentication/CI** : Gitleaks ignore ses propres faux positifs via `.gitleaksignore` ou `#gitleaks:allow`. Notez que le GitHub Action officiel

(v2) requiert une configuration YAML différente qu'en v1. Consultez sa documentation pour la version utilisée.

- *Pratiques préventives* : Ne pas exposer de clés dans les templates. Utiliser des token légitimes provisoires si nécessaire (bodied). Mettre en place des hooks Git locaux pour pré-scanner (ex : avec gitleaks protect).

## Checkov – Diagnostic et résolutions

- **Invite interactive de mise à jour (« Level up? (y/n) ») bloquant la CI**

*Message d'erreur :*

```
vbnet
```

```
Copy
```

```
Update available X.Y.Z -> A.B.C.
```

```
Would you like to "level up" your Checkov powers... To Level-up, press 'y'...
```

```
Level up? (y/n): EOFError: EOF when reading a line
```

```
` `` `[object Object].
```

*\*Cause\** : Checkov affiche par défaut une invite interactive de suggestion d'upgrade (level-up). Dans un environnement CI non interactif, cette invite génère une erreur EOF et bloque le job[object Object].

*\*Solution\** : indiquer explicitement le fichier de configuration `--config-file` sur la ligne de commande[object Object]. Exemple :

```
` `` `bash
```

```
checkov --config-file .checkov.yaml
```

Cela empêche Checkov d'utiliser son réglage par défaut et d'afficher l'invite. (On peut aussi exécuter Checkov dans son conteneur Docker officiel qui contient la dernière version sans invite.)

*Bonne pratique* : Dans un pipeline CI/CD, toujours passer `--config-file` pointant sur un fichier `.checkov.yaml` en workspace, ou désactiver les invites. On peut aussi utiliser `-q/--quiet` pour ne pas afficher les messages non critiques.

- **Erreur de parsing du plan Terraform (fichier JSON)**

*Message d'erreur :*

```
csharp
```

```
Copy
```

```
Parsing errors: 1 – Error parsing file tf.json
```

ou pas d'erreurs génériques mais absence de checks pour le plan.

*Cause* : Par défaut Checkov interprète un fichier .json comme du CloudFormation. Un plan Terraform JSON déclenchera une erreur de parsing.

*Solution* : forcer le framework Terraform lors du scan. Par exemple:

```
bash
```

```
Copy
```

```
checkov -f tfplan.json --framework terraform_plan
```

ou utiliser l'intégration [Checkov Terraform Plan Scan] (ou Terragrunt). Il faut d'abord générer un JSON lisible par Checkov :

```
bash
```

```
Copy
```

```
terraform plan -out=tfplan.binary
```

```
terraform show -json tfplan.binary > tfplan.json
```

```
checkov -f tfplan.json --framework terraform_plan
```

*Exemple CI* : dans GitHub Actions, on peut utiliser bridgecrewio/checkov-action en spécifiant -f tfplan.json et/ou framework. Par exemple.

*Prévention* : Générer le plan en JSON et toujours passer --framework terraform\_plan pour éviter ce cas.

- **Options et flags supprimés en Checkov 3.x**

Depuis Checkov 3.0, certains flags ont été retirés ou renommés. Par exemple dans la version 3.0.6, les options --no-guide, --skip-suppressions, --skip-fixes ont été supprimées et unifiées sous --skip-download.

*Impact* : Les scripts CI/CD ou alias utilisant les anciens flags échoueront.

*Solution* : utiliser la nouvelle option --skip-download à la place des anciennes, ou mettre à jour la syntaxe. Vérifiez le log de Checkov pour les suggestions de remplacement.

*Prévention* : Consulter la [doc de migration Checkov 2→3](#) avant la mise à jour majeure. Toujours pinner la version de Checkov en CI (ex. pip install checkov==3.x).

- **Exit code 0 malgré une erreur interne**

*Problème* : Quand Checkov rencontre une erreur interne (parsing ou autre bug), il peut retourner 0. Par exemple, un bug connu sur le parsing d'un chart Helm retourne exit code 0, car Checkov traite ces erreurs comme des "soft failures".

*Impact* : La CI peut passer sans signaler l'erreur réelle.

*Solution* : Aucune correction immédiate dans la version courante : il s'agit d'un

comportement par défaut. On peut cependant contourner en surveillant la sortie de Checkov. Par exemple, utiliser `--soft-fail` et `--hard-fail-on` pour contrôler finement les exit codes (voir [doc Checkov](#)). Vérifier les logs (“[ERROR]” ou exceptions) pour détecter les problèmes.

*Avenir* : Un ticket GitHub (#7393) réclame de différencier l’exit code sur erreur interne. En attendant, vous pouvez exécuter Checkov avec `-s` (soft-fail) puis vérifier manuellement le code de retour du script selon les cas.

- **Autres points de configuration et intégration CI**

- **Baselines et suppressions** : Utilisez la fonctionnalité de baseline (`--create-baseline` et `--baseline`) pour ignorer les findings existants en production.
- **Formats de sortie** : Checkov génère JSON, JUnit, SARIF, etc. Exemple SARIF pour GitHub : `checkov -d . -o sarif -s`.
- **Intégrations** : De nombreuses plateformes CI (GitLab CI, Azure DevOps, Jenkins) ont des plugins ou intégrations Checkov. Suivre les exemples officiels (ex. [\[docs Checkov GitLab CI\]](#) ou [\[HashiCorp Discuss\]](#) pour CodeBuild).
- **Prévention** : Épingler la version de Checkov, tester localement avec `checkov -v`, vérifier que Python 3.8+ est utilisé. Empêcher les invites interactives (utiliser `--ci` si disponible) et bien configurer les options `--soft-fail-on/--hard-fail-on` pour que l’exit code reflète l’état voulu.